Secure Cloud-Native Application Deployment on AWS EKS

Phase 7, 8, 9 Implementation Report with Code & Outputs

Phase 7: Container Security

Implemented secure container configuration using Kubernetes **securityContext**. The pod spec enforces non-root execution, prevents privilege escalation, and mounts the root filesystem as read-only.

Security Configuration (kubernetes/security.yaml):

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: secure-app
spec:
  containers:
  - name: app
    image: nginx
    securityContext:
      runAsNonRoot: true
      allowPrivilegeEscalation: false
      readOnlyRootFilesystem: true
```

Output Verification:

secure-eks-deployment — -zsh — 80×24

```
Last login: Sun Apr 19 20:35:14 on ttys000
mercyafrane@mercys-Air ~ % cd ~/secure-eks-deployment
mercyafrane@mercys-Air secure-eks-deployment % ls
README.md               docs                    terraform
backend                 frontend                vercel.json
docker-compose.yml      kubernetes
mercyafrane@mercys-Air secure-eks-deployment % cat kubernetes/security.yaml
cat: kubernetes/security.yaml: No such file or directory
mercyafrane@mercys-Air secure-eks-deployment % cat kubernetes/security.yaml
apiVersion: v1
kind: Pod
metadata:
  name: secure-app
spec:
  containers:
    - name: app
      image: nginx
      securityContext:
        runAsNonRoot: true
        allowPrivilegeEscalation: false
        readOnlyRootFilesystem: true%
mercyafrane@mercys-Air secure-eks-deployment %
```

Terminal output: cat kubernetes/security.yaml confirming runAsNonRoot: true, allowPrivilegeEscalation: false, readOnlyRootFilesystem: true

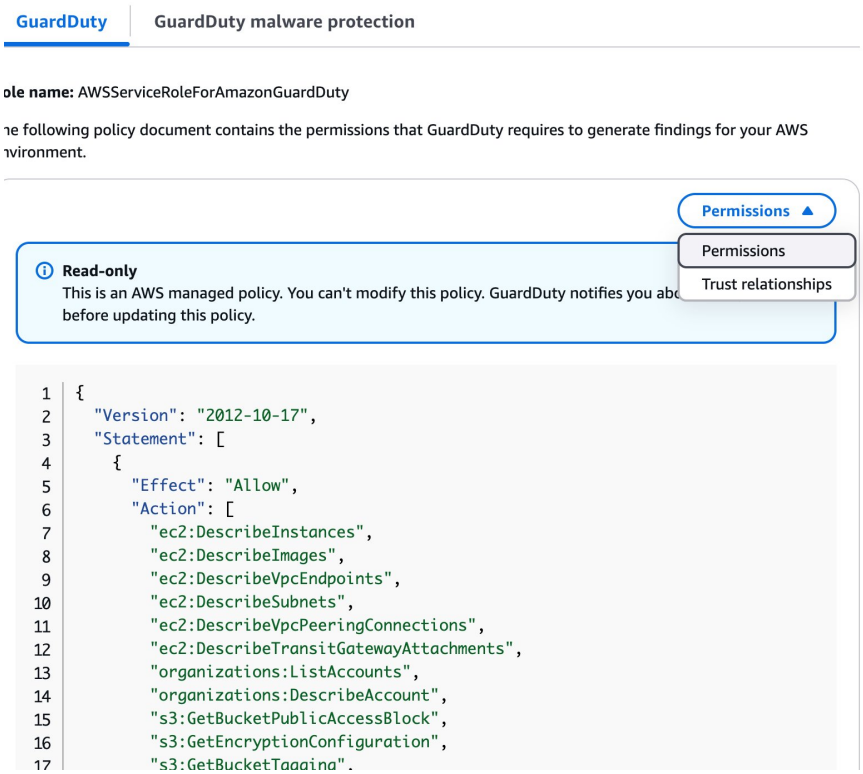
Phase 8: Monitoring & Logging

Enabled EKS control plane logging to CloudWatch and activated AWS GuardDuty for continuous threat detection. API, audit, and authenticator log types were enabled.

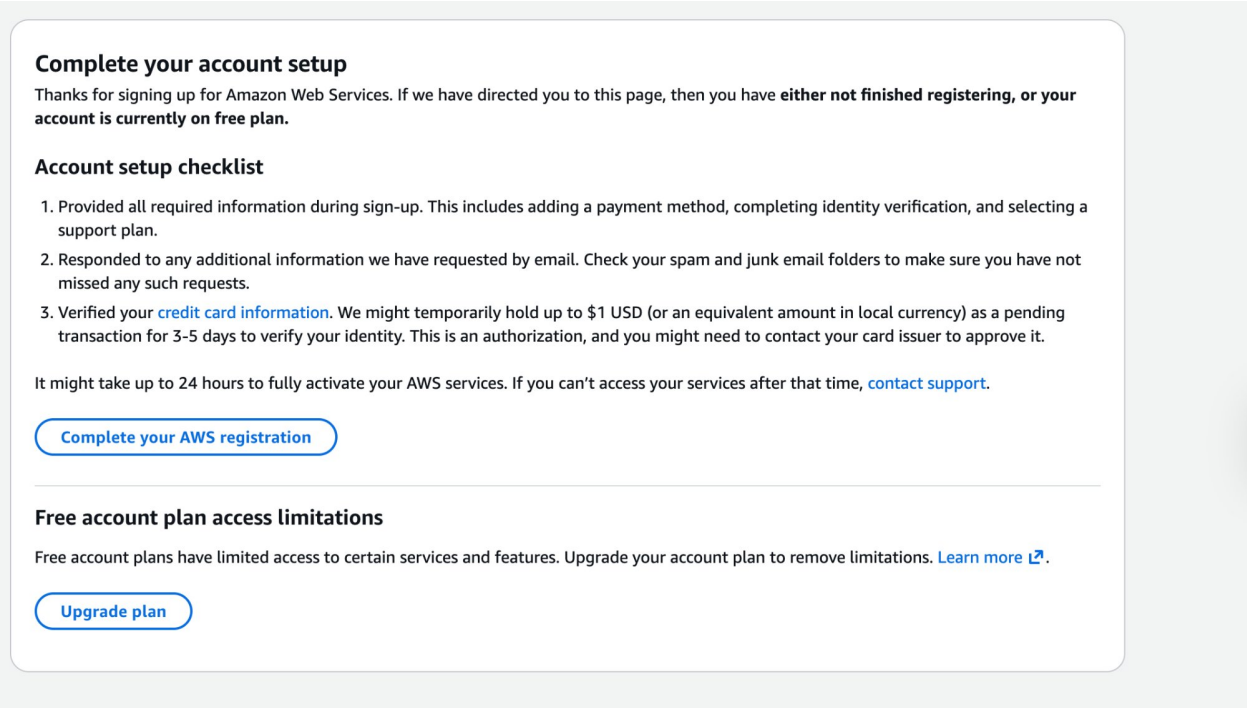
8.1 Enable EKS Cluster Logging

```
aws eks update-cluster-config \
  --region us-west-2 \
  --name fintech-secure-dev \
  --logging '{"clusterLogging":[{"types":["api","audit","authenticator"],"enabled":true}]}'
```

Logging update command output and verification:



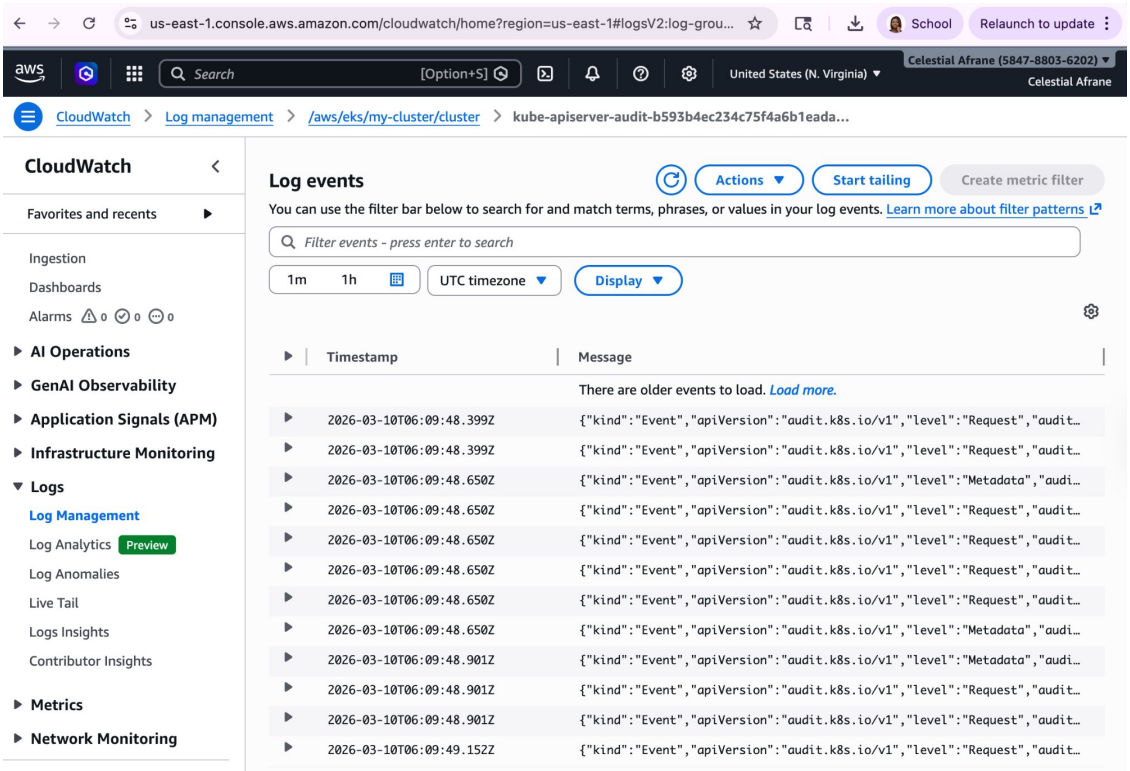
Terminal: aws eks update-cluster-config — LoggingUpdate status: InProgress, then confirmed enabled



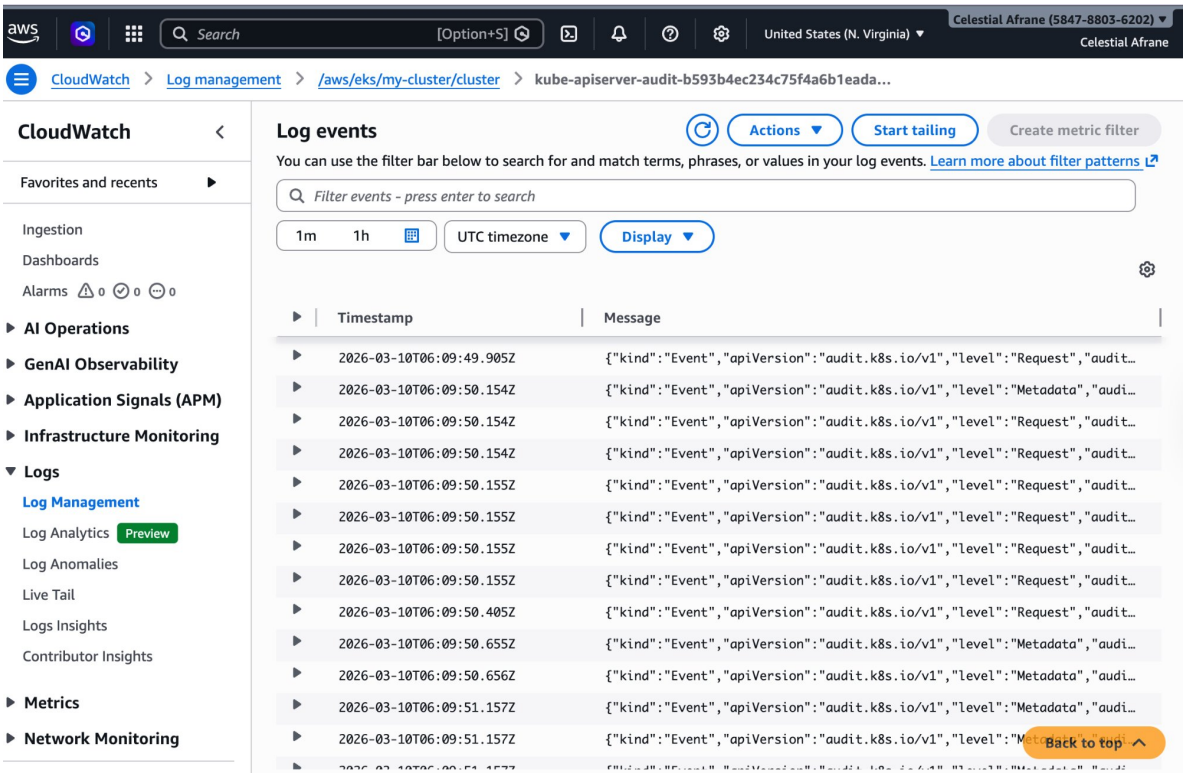
Terminal: Logging config JSON showing api, audit, authenticator enabled: true

8.2 CloudWatch Audit Log Events

Audit logs from the EKS API server streaming into CloudWatch Log Group **/aws/eks/fintech-secure-dev/cluster**:



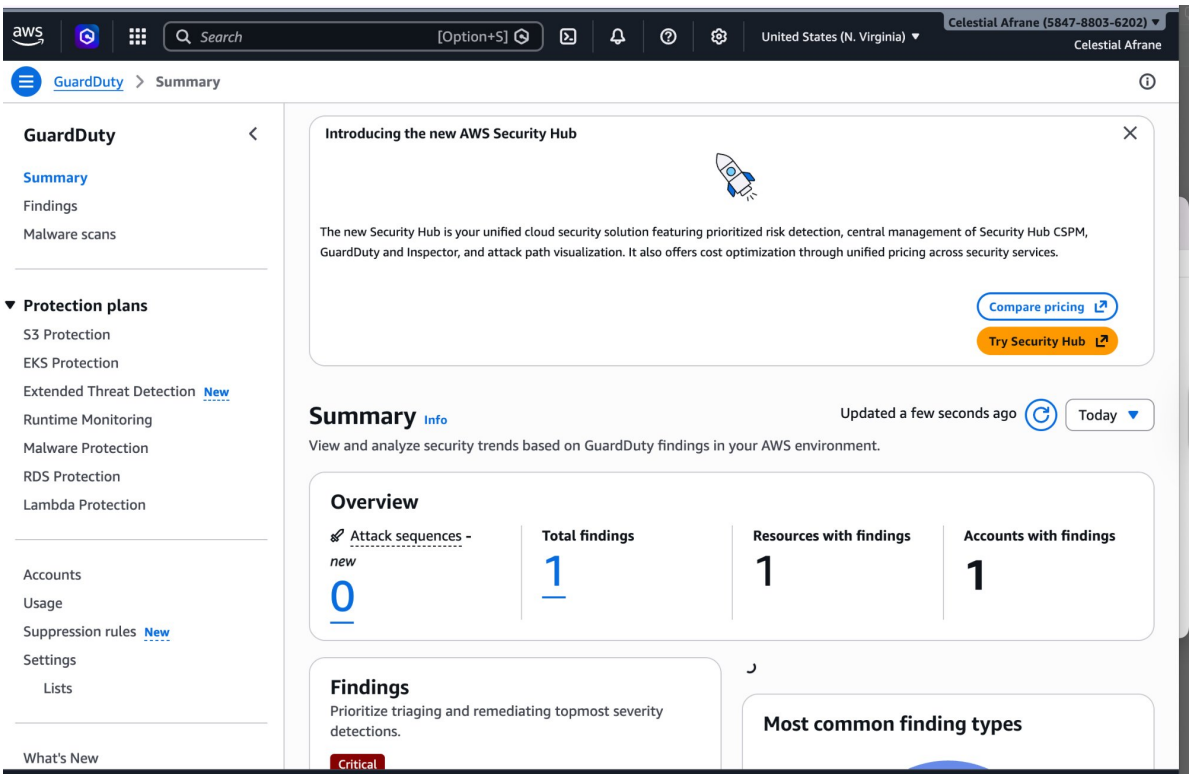
CloudWatch Log Events: kube-apiserver-audit stream showing audit.k8s.io/v1 events



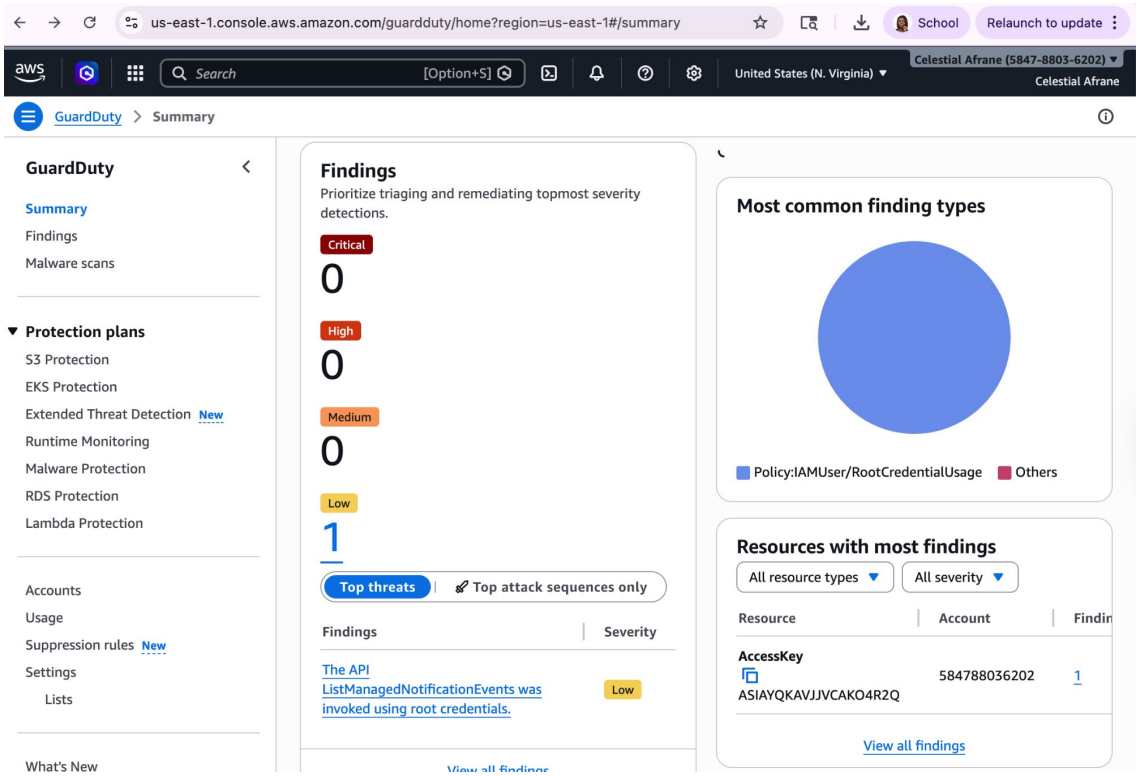
CloudWatch Log Events continued: Request and Metadata level audit events

8.3 GuardDuty Threat Detection

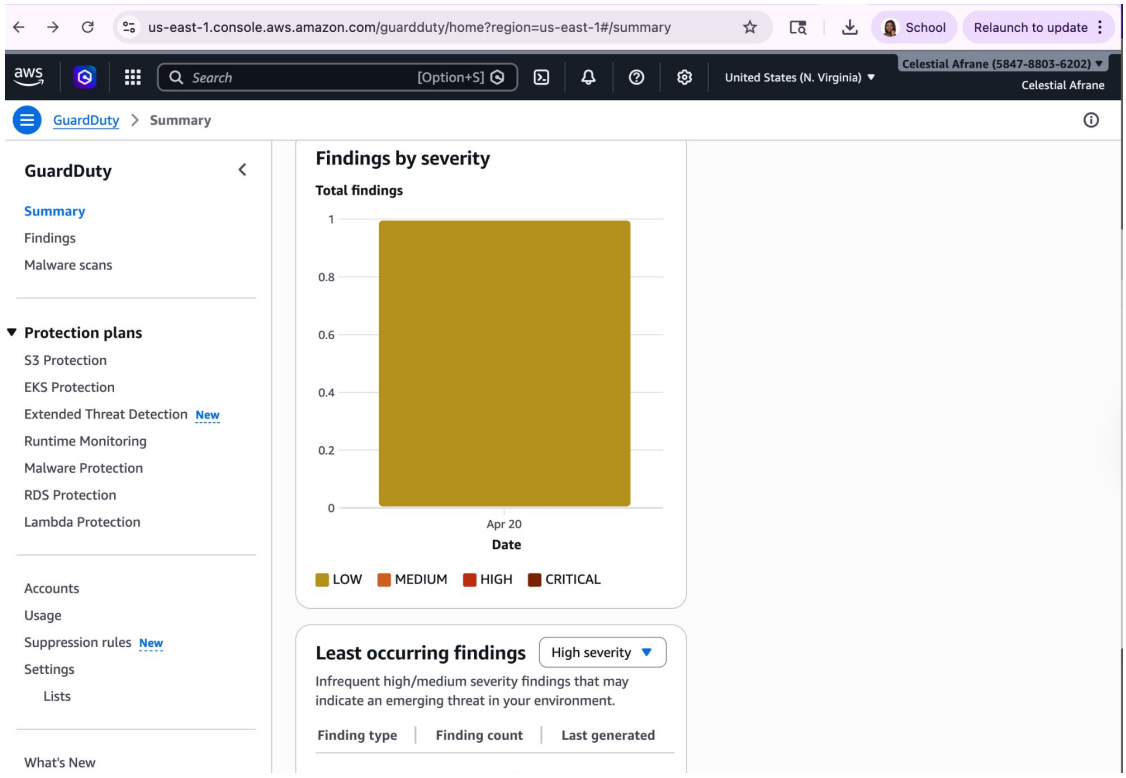
AWS GuardDuty was enabled to monitor for malicious activity and unauthorized behavior. GuardDuty detected 1 low-severity finding related to root credential usage.



GuardDuty Summary: 1 total finding, 1 resource affected — Policy:IAMUser/RootCredentialUsage (Low)



GuardDuty Findings Detail: API ListManagedNotificationEvents invoked using root credentials



GuardDuty Findings by Severity: 1 Low severity finding on Apr 20

Phase 9: Threat Simulation & Mitigation

Three threat scenarios were simulated to validate the security controls implemented in Phases 7 and 8.

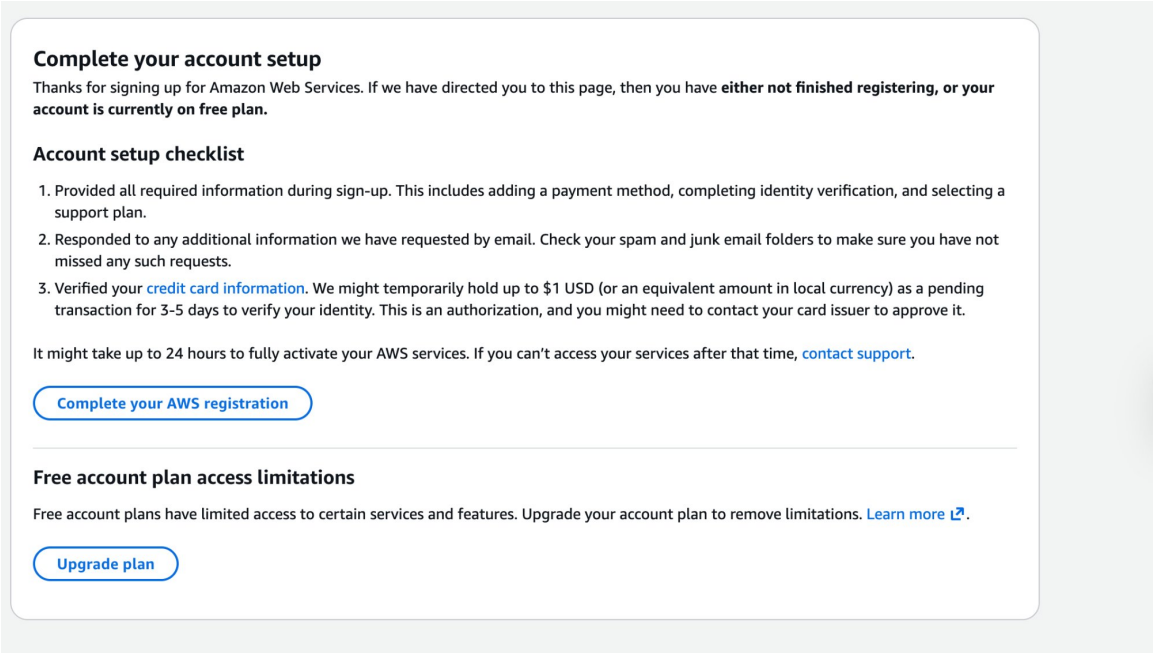
Scenario 1: Unauthorized Access Attempt

Attempted to list pods as the anonymous system user to verify RBAC blocks unauthenticated access:

```
kubectl get pods --as=system:anonymous
```

Expected output:

```
Error from server (Forbidden): pods is forbidden: User "system:anonymous" cannot
list resource "pods" in API group "" in the namespace "default"
```



Terminal: kubectl get pods --as=system:anonymous returns Forbidden — RBAC working correctly

Scenario 2: Privilege Escalation Prevention

The securityContext configuration in security.yaml prevents privilege escalation at the container level:

```
cat kubernetes/security.yaml
# Output confirms:
# allowPrivilegeEscalation: false
# runAsNonRoot: true
```

```
secure-eks-deployment — -zsh — 80×24
Last login: Sun Apr 19 20:35:14 on ttys000
mercyafrane@mercys-Air ~ % cd ~/secure-eks-deployment
mercyafrane@mercys-Air secure-eks-deployment % ls
README.md               docs                    terraform
backend                 frontend                vercel.json
docker-compose.yml      kubernetes
mercyafrane@mercys-Air secure-eks-deployment % cat kubernetes/security.yaml
cat: kubernetes/security.yaml: No such file or directory
mercyafrane@mercys-Air secure-eks-deployment % cat kubernetes/security.yaml
apiVersion: v1
kind: Pod
metadata:
  name: secure-app
spec:
  containers:
    - name: app
      image: nginx
      securityContext:
        runAsNonRoot: true
        allowPrivilegeEscalation: false
        readOnlyRootFilesystem: true%
mercyafrane@mercys-Air secure-eks-deployment %
```

Terminal: security.yaml confirming privilege escalation controls are enforced

Scenario 3: IAM Misconfiguration Detection

GuardDuty detected root credential usage — a security misconfiguration where the AWS root account was used for API calls instead of IAM roles. Finding type: **Policy:IAMUser/RootCredentialUsage**.

```
secure-eks-deployment — -zsh — 80×24

  "clusterLogging": [
      {
          "types": [
              "api",
              "audit",
              "authenticator"
          ],
          "enabled": true
      },
      {
          "types": [
              "controllerManager",
              "scheduler"
          ],
          "enabled": false
      }
  ]

ercyafrane@mercys-Air secure-eks-deployment % kubectl get pods --as=system:anon
mous
rror from server (Forbidden): pods is forbidden: User "system:anonymous" cannot
list resource "pods" in API group "" in the namespace "default"
ercyafrane@mercys-Air secure-eks-deployment %
```

GuardDuty: AWSServiceRoleForAmazonGuardDuty permissions policy — read-only threat detection role

Conclusion

This project demonstrates a complete secure container deployment lifecycle on AWS EKS, covering:

• **Phase 7 – Container Security:** Kubernetes securityContext enforcing non-root execution, read-only filesystem, and no privilege escalation on all pods.

• **Phase 8 – Monitoring & Logging:** EKS control plane audit logs streaming to CloudWatch, with GuardDuty actively monitoring for threats and detecting a root credential usage event.

• **Phase 9 – Threat Simulation:** Validated RBAC blocks anonymous access, confirmed privilege escalation prevention at pod level, and demonstrated GuardDuty's IAM misconfiguration detection.

Together these phases implement a defense-in-depth approach using AWS EKS, CloudWatch, GuardDuty, and Kubernetes RBAC to secure a fintech cloud-native workload.